

多尺度 Q-learning 风电场协同控制

理论框架与方法论

XXX

Cluing

2026 年 8 月 5 日

基于 WFCRL (风机多智能体强化学习控制) 与多尺度随机逼近理论

目录

- 1 背景与初步设定
- 2 理论框架：Assumptions & 奖励设计 & Bellman 递归
- 3 方法论：NetworkMQL
- 4 物理驱动总结与改进方向
- 5 总结与展望

风电场控制的核心问题：尾流效应

物理挑战：尾流效应 (Wake Effect)

- 上游风机提取动能 $\Rightarrow$ 下游形成低速 + 高湍流尾流区
- 后果：
 - 发电量损失：大型海上风电场 10–20%
 - 疲劳载荷增加 5–15% $\Rightarrow$ 缩短寿命

控制手段：偏航转向 (Wake Steering)

- 偏航角 $y_i \neq 0$ 横向偏转尾流
- 核心权衡：牺牲单机发电 $\rightarrow$ 提升全场总发电

核心难点： 所有偏航角到流场的映射由 3D Navier–Stokes 方程支配，无法在线优化求解。

风机控制层级与 Related Work

IEC 61400 控制层级：

层级	控制对象	时间尺度	目标
1	变桨 / 转矩	毫秒级	转速/功率调节
2	偏航 (单机)	秒-分钟级	风向追踪
3	协同偏航	分钟-小时级	全场优化

Related Work 定位：

方法	代表工作	局限性
静态偏航优化 MARL (MAPPO) 代理模型 + 贝叶斯优化 本文方法	Fleming et al. (2014) Dong et al. (2021) Kheirabadi et al. (2022) WFCRL + 多尺度 SA	无动态自适应 无收敛性保证 代理误差累积 可证明收敛 + DAG 压缩

WFCRL 基准环境与 Dec-POMDP 建模

WFCRL (NeurIPS 2024 Benchmark):

- 风电场多智能体强化学习标准基准环境
- 支持双仿真器: FLORIS (快速工程模型) + FAST.Farm (高保真物理)
- 内置功率奖励 + 疲劳惩罚

Dec-POMDP 数学建模:

$$\{M, S, \mathcal{O}, \mathcal{A}, P, o^1, \dots, o^M, r\}$$

- 全局状态 s , 局部观测 o^i , 局部动作 a^i
- 全局共享奖励 $r(s, a, s') = \text{全场功率} - \text{疲劳惩罚}$
- 转移核 $P(s, a, s')$ 由风场物理 (N-S) 决定

关键转化——Transition Independence:

$$(y_i, w_i) \longrightarrow (y_i, w_\infty)$$

- 将耦合的本地风况 w_i 替换为共享的外界来流 w_∞
- 牺牲状态可观测性 $\Rightarrow$ 换取转移独立性

DAG 结构：将物理先验编码进算法

物理事实：风只能从上游吹向下游 $\Rightarrow$ 尾流影响是有向无环的。

DAG 定义：

$$\mathcal{G} = (V, E), \quad i \rightarrow j \iff \text{风机 } j \text{ 在风机 } i \text{ 的尾流影响范围内}$$

对 RL 算法的意义（三重物理注入）：

- ① 因果排序：上游 $\rightarrow$ 下游，决定学习先后
- ② 尺度分配：祖先（上游）收敛更慢，后代（下游）收敛更快
- ③ 信息局部化：每台风机只需与上游邻居通信

示例（ 2×2 风场，风向从左向右）：

$$1 \rightarrow \{2, 3, 4\}, \quad 2 \rightarrow 4, \quad 3 \rightarrow 4$$

该 DAG 是 *Algorithm 1*（学习率等级分配）的输入骨架。

理论框架总览（四大 Assumptions）

为证明多智能体 Q-learning 在风电场场景下能稳定收敛，论文引入了四个核心假设：

假设	核心内容	一句话直觉
Assumption 1	局部转移独立分解	“每台风机的偏航只跟自己有关”
Assumption 2	遍历性	“长期运行后能忘掉初始状态”
Assumption 3	多尺度学习率	“上游慢学，下游快学”
Assumption 4	奖励 Lipschitz 压缩	“尾流耦合不能太强”

四个假设共同保证：算法最终收敛到“平滑纳什均衡”。

Assumption 1: 局部转移独立性

数学定义：

$$P(s, a, s') = \prod_{i=1}^M P^i(s_i, a_i, s'_i)$$

大白话：

- 全局状态 s 从当前时刻到下一时刻的转移概率，可以分解成每台风机自己转移概率的乘积。
- 也就是说：**风机 i 的下一时刻状态只取决于它自己的当前状态和动作**，与其他风机无关。

为什么在风电场中成立？

- 偏航角的执行是本地闭环控制——我发指令 a_i ，它转 a_i 度。
- 上游风机动不动，不影响我自己能不能转到目标角度。

没有它会怎样？

- 每个风机的状态转移都要依赖所有其他风机的动作 $\Rightarrow$ 联合状态空间爆炸，无法计算。
- 这个假设是 TI-Dec-POMDP 的定义性特征，是整个框架的基石。

Assumption 2: 遍历性 (Ergodicity)

数学定义 (非正式版):

- 对任何“非退化”局部策略 (所有动作被选中的概率都 > 0), 局部状态过程是:
 - ① 不可约 (Irreducible): 从任何状态都能到达任何其他状态
 - ② 非周期 (Aperiodic): 不会只在固定的几步循环里打转

大白话:

- 只要你让风机偶尔也试试非最优动作 (探索), 那么:
 - 无论今天偏航角是多少, 长期跑下去, 访问各个偏航角的频率是固定的;
 - 这个固定频率与“今天开机时是 0° 还是 20° ”无关。
- 这个固定频率就是平稳分布 d_π (后面会用到)。

为什么需要它?

- 如果状态过程不遍历, 长期平均奖励会依赖初始状态——不同风机从不同角度出发, 算出的“平均表现”不一样, 没法统一评估。
- 而且 RL 采样的样本必须有代表性, 遍历性保证了你采的样本能覆盖所有状态。

Assumption 3: 多尺度学习率 (Multi-Scale Learning Rates)

三组条件:

$$(RM) \quad \lim_k \alpha_k^i = 0, \quad \sum_k \alpha_k^i = \infty$$

$$(Slow \ Change) \quad \lim_{n \rightarrow \infty} \sup_{0 \leq j \leq a_n^i} \left| \frac{\alpha_{n+j}^i}{\alpha_n^i} - 1 \right| = 0$$

$$(Separation) \quad \frac{\alpha_k^i}{\alpha_k^j} \rightarrow 0 \quad (i < j)$$

逐条人话解释:

- ① **RM 条件:** 学习率最终趋近于 0 (不会震荡), 但所有学习率加起来是无穷大 (不会半途停下)。——标准 SGD 也有这条。
- ② **慢变条件:** 学习率在局部窗口内变化得极慢。——因为 RL 的数据是时序相关的 (马尔可夫链), 需要足够时间让噪声链充分混合。标准 SGD 的平方可和条件在这里不够用。
- ③ **分离条件:** $i < j$ 时, 第 i 个智能体的学习率比第 j 个智能体小得多, 所以 i 收敛得更快 (步长小但衰减快), j 收敛得更慢 (步长大但衰减慢)。

物理映射: 上游风机 (影响大) $\rightarrow$ 慢收敛; 下游风机 (影响小) $\rightarrow$ 快收敛。

Assumption 4: 奖励 Lipschitz 压缩

数学定义:

$$\|R_\pi - R_{\pi'}\|_1 \leq K \|Q_i - Q'_i\|_\infty, \quad K \in (0, 1)$$

人话解释:

- 如果你把某一台风机 i 的 Q 表改动了一下 (变化量记为 $\|Q_i - Q'_i\|$),
- 那么这个改动通过尾流耦合传播后, 最终导致的全场稳态奖励的变化 ($\|R_\pi - R_{\pi'}\|$),
- 被严格压缩了 (乘以一个小于 1 的系数 K)。

为什么需要它? ——收敛条件的源头

- 在标准 Q-learning 中, Bellman 算子是 β -收缩的 ($\beta < 1$), 所以一定收敛。
- 在多智能体场景中, 别人的策略也在变, Bellman 算子**不一定**收缩。
- 但如果尾流耦合本身是压缩的 ($K < 1$), 那么**联合更新算子的收缩系数** = $K + \beta$ 。
- 因此要求 $\beta \leq 1 - K$, 即 $K + \beta \leq 1$, 才能保证收敛。

这本质上是 *Bellman* 收缩定理在多智能体场景下的推广。

奖励设计（一）：功率奖励

公式：

$$r_k^P = \frac{1}{M} \sum_{i=1}^M \frac{\hat{P}_k^i}{(u_{\infty,k})^3}$$

为什么要除以 u_{∞}^3 ？

- 风能通量 $P \propto u_{\infty}^3$ （物理常识）。
- 如果不归一化，10 m/s 时的功率可能是 5 m/s 时的 8 倍，RL 智能体会误以为“高风速 = 自己表现好”。
- 除以 u_{∞}^3 后，奖励变成一个无量纲的效率系数，只反映“控制动作有没有缓解尾流损失”。

直观类比：

- 不归一化 = 考试分数受“试卷难易度”影响，无法比较不同学生的真实水平。
- 归一化后 = 试卷标准化，分数只反映学生能力。

目的：让学习信号反映“控制质量”，而不是“今天风多大”。

奖励设计（二）：疲劳惩罚（双版本）

版本 1：FLORIS 代理惩罚（快速，用于训练）

$$r_{k,S}^L = \frac{1}{M} \sum_{i=1}^M \left(\sum_{j=1}^9 Tl_k[x_{i,j}, y_{i,j}] + \sigma(u_k) + \sigma(v_k) \right)$$

- FLORIS 没有结构动力学模型，所以用流场统计量代理疲劳：
 - 湍流强度 TI （平均应力源）
 - 流向速度标准差 $\sigma(u)$ （挥舞弯矩的波动）
 - 横向速度标准差 $\sigma(v)$ （摆振弯矩的波动）
- 9 点网格 = 3×3 采样转子平面，在精度与速度间取得平衡。

版本 2：FAST.Farm 物理惩罚（高保真，用于评估/微调）

$$r_{k,D}^L = \frac{1}{M} \sum_{i=1}^M \left(\sum_{j=1}^3 |M_k^{op}[i,j]| + \sum_{j=1}^3 |M_k^{ip}[i,j]| \right)$$

- 直接使用叶片挥舞弯矩（面外）和摆振弯矩（面内）的绝对值之和。
- **取绝对值**：因为正负弯矩方向相反，但疲劳损伤只看幅值，不能相互抵消。

Bellman 递归 (一): 局部 Q 函数的定义

为什么要定义“局部” Q 函数？

- 在单智能体 RL 中，Q 函数是 $Q(s, a)$ 。
- 但在多智能体场景中，你的奖励受别人影响，所以必须区分：
 - 你自己的策略 π_i
 - 别人的策略 π_{-i}

局部 Q 函数的定义：

$$Q_{\pi_i}^{\pi_{-i}}(s_i, a_i) = \mathbb{E}_{s_0 \sim d_\pi, a_k \sim \pi, s_k \sim P} \left[\sum_{k=0}^{\infty} \beta^k r(s_k, a_k) \mid s_0^i = s_i, a_0^i = a_i \right]$$

人话翻译：

- “在风电场长期运行稳定后（从平稳分布 d_π 出发），
- 如果风机 i 当前偏航角是 s_i ，我尝试了一个动作 a_i ，
- 之后所有风机（包括我）都按既定策略行事，
- 那么我未来拿到的折扣奖励总和的期望是多少？”

Bellman 递归 (二): 局部 Bellman 方程

公式:

$$Q_{\pi_i}^{\pi_{-i}}(s_i, a_i) = \underbrace{\sum_s d_{\pi}(s|s_i) \sum_{a_{-i}} \pi_{-i}(a_{-i}|s) r(s, a)}_{\text{即时奖励项 (依赖别人策略)}} + \beta \underbrace{\sum_{s'_i} P^i(s_i, a_i, s'_i) \sum_{a'_i} \pi_i(a'_i|s'_i) Q_{\pi_i}^{\pi_{-i}}(s'_i, a'_i)}_{\text{未来项 (只依赖自己)}}$$

与标准 Q-learning 的两大区别:

- ① 即时奖励项: 需要对所有其他风机 a_{-i} 的动作取期望, 因为全局奖励是共享的。
 - 问题: 你看不到别人在干什么, 只能看到一个采样出来的具体奖励。
 - 这个差异就是 **非平稳性 (non-stationarity)** 的来源。
- ② 未来项: 状态转移 P^i 只依赖局部状态和动作 (Assumption 1), 所以这块跟单智能体一样。

这个方程把“别人的影响”全部压缩到了即时奖励项里。

平滑最优响应 (Smoothed Best Response) 与均衡

为什么要“平滑”？

- 如果直接用 $\arg \max$, 策略是确定性的 (最优动作概率 = 1, 其他 = 0)。
- 这会破坏 Assumption 2 (遍历性) —— 有些状态-动作对永远不被探索。

熵正则化最优响应 (Softmax 风格的 Q 值映射):

$$\phi(Q_i)(\cdot | s_i) = \arg \max_{\rho \in \Omega(s_i)} [\rho^\top Q_i(s_i, \cdot) + \tau \nu_{s_i}^i(\rho)]$$

- $\Omega(s_i)$: 动作空间上的概率分布 (所有动作概率之和为 1)
- $\tau > 0$: 温度系数 (τ 越大, 策略越随机)
- ν : 强凹正则项 (保证解唯一)
- 效果: Q 值高的动作概率大, 但所有动作概率都 > 0

均衡定义 (Definition 2):

全局策略 π^* 是均衡 $\iff$ 每个 $\pi_i^* = \phi(Q_i^{\pi^*})$ 是对其他策略的平滑最优响应。

即: 没有任何风机能通过单方面改变策略来获得更高回报。

DAG 等级分配：为什么要压缩时间尺度？

问题：

- 如果每台风机分配一个独立的时间尺度， M 台风机就需要 M 个不同的学习率衰减曲线。
- 16 台还能忍，100 台就太浪费了。

核心思想：利用 DAG 结构压缩

- 在 DAG 中，**同一层（互不影响）**的风机可以共享同一个时间尺度。
- 只需要 **图的深度**个尺度，而不是 M 个。

两个硬约束（保证收敛）：

- ① **条件 A（祖先优先）**：上游风机的等级必须比下游风机**更小**（即收敛更慢）。
- ② **条件 B（无共享后代）**：同一等级的两个风机**不能有共同的后代**。
 - 否则下游会同时面对两个“还在变化”的上游源，无法稳定学习。

论文报告：16 台风机通过 *Algorithm 1* 压缩到 9 个尺度。

Algorithm 1: 等级分配算法（人话版）

输入：尾流影响 DAG $\mathcal{G} = (V, E)$

算法步骤（非代码，讲逻辑）：

- ① 找末端风机（Leaves）：最下游、不受别人尾流影响的风机。
- ② 反向搜索约束集：对每个末端风机，向上找它所有的祖先（上游风机），把这些祖先放进一个集合 Σ 。这些祖先绝对不能有相同的等级。
- ③ 拓扑排序：从最上游开始，依次分配等级。
- ④ 等级分配规则：
 - 最上游（无父节点） $\rightarrow$ 等级 0（最慢收敛）
 - 其他节点 $\rightarrow$ 父节点最大等级 + 1
 - 如果这个等级和约束集 Σ 冲突了，就强制 +1，直到不冲突为止。

输出：每台风机的等级 $d_i \in \{0, 1, \dots, \bar{M} - 1\}$, $\bar{M} \leq M$ 等级越大，收敛越快（学习率衰减越慢）。

多尺度学习率设计（从等级到数值）

基础版本（基于访问次数）：

$$\alpha_{k,c}^i = \frac{g}{n_k((s_i, a_i)_c)^{l_i}}, \quad 0 < l_M < \dots < l_1 < 1$$

- n_k : 状态-动作对 (s_i, a_i) 被访问的次数
- l_i : 风机 i 的等级指数（由 Algorithm 1 决定）
- 等级越大 $\Rightarrow l_i$ 越小 $\Rightarrow$ 分母增长越慢 $\Rightarrow$ 学习率衰减越慢 $\Rightarrow$ 收敛越快

含实时修正项的实际版本（防止 Q 表条目爆炸）：

$$\alpha_{k,c}^i = g \left(\frac{1}{n_k} + \frac{\log(T_{k,c}^i) - \log(T_{k-1,c}^i)}{T_{k,c}^i - T_{k-1,c}^i} \right)^{l_i}$$

- $T_{k,c}^i$: 组件 c 第 k 次更新的真实时间
- 作用：防止长期未访问的 Q 表条目在突然更新时产生数值跳跃。

NetworkMQL 更新规则

Step 1: 局部奖励分解 (ND-POMDP 结构)

$$r(s, a) = \sum_{i=1}^M r_i(s_i, a_i, s_{u_i}, a_{u_i})$$

- 风机 i 的局部奖励只依赖自己和上游邻居 u_i 。
- 实际更新时使用邻居奖励和: $\bar{r}_k^i = \sum_{j \in \{i, \mathcal{N}_i\}} r_j$
- 通信开销大幅降低 (只需与邻居通信, 不需全局广播)

Step 2: 局部 Q 表更新

$$\hat{Q}_{k+1,c}^i = \hat{Q}_{k,c}^i + \alpha_k^i(s_k^i, a_k^i) \left[\bar{r}_k^i + \beta \left(\phi(\hat{Q}_k^i)(s_{k+1}^i)^\top \hat{Q}_k^i(s_{k+1}^i, \cdot) - \hat{Q}_{k,c}^i \right) \right]$$

- TD 目标使用 $\phi(Q_i)$ (平滑策略的期望), 而不是贪心 $\max$
- 既保证了探索 (所有动作概率 >0), 又保留了 Q 值引导

收敛性定理 (Theorem 2 & 3)

Theorem 2 (多尺度 Q-learning 收敛):

- 若 Assumption 4 成立 ($K < 1$) 且折扣因子满足 $\beta \leq 1 - K$,
- 则 Q 值估计弱收敛到平滑最优响应值 Q^{*i} ,
- 诱导策略 $\pi^{*i} = \phi(Q^{*i})$ 是一个均衡。

Theorem 3 (NetworkMQL 收敛):

- 在 DAG 分层满足条件 A (祖先优先) 和条件 B (无共享后代) 的前提下,
- NetworkMQL 保留 Theorem 2 的全部收敛性结论,
- 但只需 $\bar{M} \leq M$ 个时间尺度。

经验验证:

- 16 台风场仿真: NetworkMQL 收敛, naive Q-learning (所有风机同一尺度) 发散。
- 论文报告 $\bar{M} = 9$ (从 16 压缩到 9)。

物理信息如何注入算法？——五层结构

这篇论文的“物理驱动”不是把 N-S 方程硬编码进神经网络，而是通过“结构设计”和“信息流约束”来体现。

层级	注入方式	对应物理事实
1. DAG 结构	算法输入	尾流只从上游到下游（有向无环）
2. 局部奖励	奖励分解	只有邻居影响你（空间局部性）
3. 时间尺度	学习率排序	上游先稳定，下游后响应（因果时序）
4. 条件 B	等级分配约束	不能同时有两个变量影响同一下游（可分辨性）
5. 确定性转移	状态更新规则	偏航执行器物理特性 ($s'_i = s_i + a_i$)

算法不求解流体方程，但信息流方向完全遵从物理方程的方向性。

物理驱动 vs 纯数据驱动

维度	本文方法（物理驱动）	纯数据驱动（naive MARL）
结构约束	DAG 显式编码上下游关系	所有风机平等对待
学习率分配	上游慢、下游快（因果驱动）	所有风机同一尺度
通信范围	仅与上游邻居通信	全局广播或全连接
收敛性	可证明收敛（Theorem 2 & 3）	可能发散或震荡
样本效率	高（利用物理先验减少探索）	低（盲目试错）
工程部署	分布式、通信轻量	需要全局协调

实验证据：

- 16 台风场仿真：NetworkMQL 收敛，naive Q-learning（所有风机同一尺度）**发散**。
- 物理先验不是“锦上添花”，而是“收敛的必要条件”。

改进点 1：疲劳精确建模

现状：

- FLORIS 代理：用湍流强度 + 速度标准差近似疲劳，精度有限。
- FAST.Farm 物理：直接使用弯矩绝对值，但未引入长期疲劳累积准则。

改进方向：引入 Miner 线性累积损伤 + DEL

- Miner 准则：疲劳损伤 $D = \sum \frac{n_i}{N_i}$ ，其中 n_i 是应力循环次数， N_i 是该应力幅值下的疲劳寿命。
- 等效疲劳载荷 (DEL, Damage Equivalent Load)：将变幅载荷折算为等效恒幅载荷，更接近真实叶片寿命评估。
- 工程价值：直接优化 DEL 而不是湍流强度，让奖励函数物理意义更明确。

可行性：

- FLORIS 无结构模型，需引入代理模型（如神经网络回归）快速估计 DEL。
- 或直接使用 FAST.Farm 做奖励计算（但速度慢，适合最终评估而非训练）。

改进点 2：连续动作空间扩展

现状：

- 动作空间为离散 $\{-1^\circ, 0, +1^\circ\}$ ，限制偏航控制的平滑性和最优性。
- 实际风机的偏航执行器支持连续角度调节（精度通常 $\pm 0.1^\circ$ ）。

改进方向：引入连续动作空间 MARL 算法

- MAPPO (Multi-Agent Proximal Policy Optimization)：适合连续动作的 on-policy 算法，稳定性好。
- MATD3 (Multi-Agent Twin Delayed DDPG)：适合连续动作的 off-policy 算法，样本效率高。

需处理的物理约束：

- 偏航执行器**最大转速** ($\sim 0.5\text{--}1.5^\circ/\text{s}$)
- 偏航**响应延迟**（机械惯性）
- 在动作空间或奖励中**惩罚超限动作**

连续动作可使每台风机更精细地调节偏航角，进一步提升全场发电量。

改进点 3: DAG 在线更新与自适应尺度分配

现状:

- DAG 基于**固定风向**预先给定。
- 实际风场风向会变化 (如从 0° 变为 30°), 尾流影响图应随之改变。

改进方向 1: 基于实时 SCADA 数据的图估计

- 利用**因果关系发现算法** (如 Granger Causality、Transfer Entropy) 从实时数据中推断尾流影响关系。
- 论文已有 TI 假设 (状态转移独立), 使得因果发现更简单——只需检测“风机 i 的偏航变化是否预测风机 j 的功率变化”。

改进方向 2: 自适应时间尺度重分配

- 当风向变化触发 DAG 拓扑更新时, **动态调整各风机的学习率等级**。
- 涉及: 检测到拓扑变化 $\rightarrow$ 重新运行 Algorithm 1 $\rightarrow$ 平滑过渡到新的学习率曲线。
- **工程价值: 使算法具备真正的“风场自适应”能力。**

改进点 4：模型基与无模型混合 (Model-Based + Model-Free)

现状：

- 纯无模型 Q-learning，需大量仿真交互（样本效率低）。
- FLORIS 工程模型虽不精确，但计算极快，可提供有效先验。

改进方向：Dyna-Style 架构

Dyna 架构三步骤

1. 真实交互：用当前策略与环境 (FAST.Farm) 交互，收集 (s, a, r, s') 存入经验池
 2. 模型学习：用经验池数据训练 FLORIS 代理模型（回归 N-S 的输入输出映射）
 3. 规划/想象：用代理模型生成大量**虚拟轨迹**，额外更新 Q 表（不消耗真实仿真时间）
-

预期收益：

- 同等样本量下，收敛速度提升 2–5 倍（类似 DQN 中经验回放 + 模型预测的结合）。
- 降低对高保真仿真器 (FAST.Farm) 的依赖。

改进点 5：通信延迟与丢包鲁棒性

现状：

- NetworkMQL 假设局部奖励通信可靠（即上游邻居的奖励信息能实时、无损到达）。
- 实际风场中，工业以太网或 5G 通信存在延迟、丢包、乱序等问题。

改进方向：延迟/丢包感知的 Q 更新

- **延迟补偿**：为每个邻居维护一个消息年龄（Message Age），在 Q 更新时根据消息时效性调整权重（旧消息权重低）。
- **丢包处理**：若某邻居信息缺失，用历史平均奖励或预测值替代（类似 Kalman Filter 的预测步）。
- **异步容忍**：当前算法本身已是异步（智能体互不等待），天然具备一定鲁棒性；在此基础上显式建模通信质量可进一步提升。

使算法更贴近实际风场的工业部署条件。

改进点 6：从表格 Q 到深度 Q（DQN 扩展）

现状：

- 当前理论基于**表格 Q-learning** (tabular)，状态空间受限于离散偏航角数量。
- 实际风场状态空间大（连续风速、风向、湍流强度），表格方法无法处理。

改进方向：将 θ^i 映射为 DQN / Duelling DQN 网络参数

- 将 Q 表替换为神经网络 $Q_{\theta^i}(s_i, a_i)$ ，输入为连续状态，输出为 Q 值。
- 保持 Assumption 1（局部转移独立）的物理结构不变。
- 将多尺度 SA 收敛性分析拓展到函数近似场景（需处理近似误差的传播）。

需解决的理论挑战：

- ① 函数近似引入的 **TD 误差传播**可能破坏收敛性。
- ② 非线性网络在马尔可夫噪声下的收敛分析更复杂。
- ③ 各智能体的网络更新仍然是异步独立的，但需要**目标网络**（Target Network）来稳定训练。

改进方向总结

改进点	核心问题	预期收益
1. 疲劳精确建模	当前疲劳代理精度有限	更真实的寿命优化
2. 连续动作空间	离散动作限制最优性	更精细的控制
3. DAG 在线更新	固定拓扑不适应风向变化	风场自适应能力
4. 模型基混合	纯无模型样本效率低	加速收敛 2-5 倍
5. 通信鲁棒性	假设通信理想	工程可部署性
6. 深度 Q 扩展	表格法状态空间受限	处理连续高维状态

优先级建议： 4（模型基混合）→ 3（DAG 在线更新）→ 6（深度扩展）→ 2（连续动作）→ 1（疲劳建模）→ 5（通信鲁棒性）

本文核心贡献总结

模块	核心内容
Preliminary Theory Reward Methodology Convergence Physical Info	TI-Dec-POMDP + DAG 物理结构注入 4 个 Assumptions 保障收敛 + Bellman 局部递归 + 平滑最优响应均衡 功率归一化 + 双版本疲劳惩罚 (FLORIS 代理 / FAST 物理) Algorithm 1 等级分配 + 局部邻居奖励 + 多尺度学习率 $\beta \leq 1 - K$ + DAG 分层保留收敛性 五层物理注入 (DAG / 局部奖励 / 时间尺度 / 条件 B / 确定性转移)

核心增量贡献：

- 首次将多尺度随机逼近理论系统性地应用于风电场 MARL 控制问题
- 在尾流 DAG 上实现了可证明收敛、可工程压缩的分布式 Q-learning 算法

研究路线图与下一步计划

已完成（理论 + 算法）：

- ① WFCRL 环境验证 + Gymnasium 接口对接
- ② TI-Dec-POMDP 建模 + DAG 物理结构编码
- ③ 四大 Assumptions + Bellman 局部递归推导
- ④ NetworkMQL 算法设计 (Algorithm 1 + 多尺度学习率)
- ⑤ 收敛性定理 (Theorem 2 & 3) 理解与证明梳理

后续规划（优先级排序）：

- ① **模型基混合 (Dyna-Style)**：用 FLORIS 代理模型加速训练
- ② **DAG 在线更新**：实现风向变化下的自适应尺度分配
- ③ **深度 Q 扩展**：将 Q 表替换为 DQN，处理连续状态空间
- ④ 在 16 台风场基准上复现论文结果，并验证改进点的有效性

参考文献

-  Bizon Monroc, C., Bušić, A., Dubuc, D., Zhu, J. *WFCRL: A Multi-Agent Reinforcement Learning Benchmark for Wind Farm Control*. NeurIPS 2024, Track on Datasets and Benchmarks.
-  Kushner, H. J., Yin, G. G. *Stochastic Approximation and Recursive Algorithms and Applications*. Springer, 2003.
-  Borkar, V. S. *Stochastic Approximation: A Dynamical Systems Viewpoint*. Cambridge University Press, 2008.
-  Goldman, C. V., Zilberstein, S. *Optimizing Information Exchange in Cooperative Multi-Agent Systems*. AAMAS 2003.
-  Fleming, P. A., et al. *Simulating the Wake Steering Effect*. J. Phys.: Conf. Ser. 2014.
-  Dong, H., et al. *Multi-Agent Reinforcement Learning for Wind Farm Control*. 2021 (preprint).
-  Kheirabadi, A. C., et al. *Gaussian Process Regression for Wind Farm Control*. 2022.
-  Zalkind, D. S., et al. *Fatigue-Aware Wind Farm Control*. 2019.